CloudCanvas: AWS SaaS Architecture

Portfolio Case Study | Architecture Design Proposal

Prepared for: Cloud Architect Career Portfolio

Status: Proposed design; implementation and performance not yet verified

Contents

1. Executive Summary
2. Business Context and Problem Statement
3. Goals and Non-Goals
4. Proposed High-Level Architecture
5. AWS Service Choices and Rationale
6. Network and Security Design
7. Application, Database, and Cache Design
8. Availability, Recovery, and Observability
9. CI/CD and Infrastructure as Code
10. Architecture Decision Records (ADRs)
11. Failure Scenarios and Responses
12. Capacity and Performance Validation
13. Cost Model and Financial Assumptions
14. Risks, Trade-offs, and Open Questions
15. Implementation Roadmap
16. Portfolio Evidence and Validation Checklist
17. Professional Reflection

1. Executive Summary

CloudCanvas is a hypothetical software-as-a-service (SaaS) platform used to demonstrate cloud architecture and systems-design skills. This case study proposes an AWS architecture intended to support secure access, horizontal application scaling, resilient data storage, and a maintainable operational model.

The proposed design uses Amazon Route 53, Amazon CloudFront where edge delivery is beneficial, AWS WAF, an Application Load Balancer (ALB), Amazon ECS on AWS Fargate, Amazon RDS for PostgreSQL in a Multi-AZ deployment, Amazon ElastiCache for suitable cache workloads, and AWS observability and security services. GitHub-based CI/CD and Terraform are proposed for repeatable delivery and infrastructure management.

Project status: architecture design proposal. This document does not claim production deployment, load testing, or achieved service-level results. Performance, availability, recovery, and cost assumptions require validation.

2. Business Context and Problem Statement

CloudCanvas represents a growing SaaS business that needs a maintainable cloud foundation as its application and customer base expand. The design aims to reduce dependence on manually managed servers, provide a repeatable deployment path, and establish clear boundaries between public entry points, application workloads, and persistent data.

The specific customer count, workload profile, business domain, and service-level objectives were not fully specified in the notes available for this reconstruction. These remain discovery items rather than invented facts.

3. Goals and Non-Goals

- Goals: establish a secure and maintainable AWS foundation; support horizontal scaling of stateless application tasks; isolate application and database tiers; improve deployment repeatability; enable monitoring and tracing; plan for availability and recovery; keep operational burden manageable.
- Non-goals at this stage: claim a verified production SLA; claim measured throughput or latency; assert that disaster recovery has been tested; or present an estimate as an actual AWS bill.

4. Proposed High-Level Architecture

Proposed request path: user → Route 53 → CloudFront (where edge delivery is useful) → AWS WAF → ALB → ECS Fargate tasks in private subnets. The application accesses PostgreSQL in database subnets and uses ElastiCache only for suitable cache/session workloads. Add background workers and asynchronous workflows only where requirements justify them.

CloudFront caching must be configured deliberately. Dynamic authenticated API responses should not be cached unless a specific endpoint is proven safe to cache and its cache key and authorization behavior are correctly designed. Static assets are the natural initial caching candidate.

Diagram to include: a VPC spanning at least two Availability Zones; public subnets for the ALB; private application subnets for ECS tasks; isolated database subnets for RDS; and the required service connections. Show NAT gateways or VPC endpoints according to the chosen egress design.

5. AWS Service Choices and Rationale

Layer	Proposed service	Rationale / validation
DNS / edge	Route 53; CloudFront	DNS and optional edge delivery; validate whether CloudFront helps the API and asset profile.
Protection / ingress	AWS WAF; ALB	Filter common web threats and distribute HTTP/S traffic to healthy targets.
Compute	ECS Fargate	Reduces host administration; validate task sizing and scaling through representative tests.
Data	RDS PostgreSQL Multi-AZ	Managed relational database with high-availability failover; standby is not normally a read-scaling endpoint.
Cache	ElastiCache	Use for suitable cacheable data; PostgreSQL remains authoritative for durable business records.
Operations	CloudWatch; X-Ray	Metrics, logs, alarms, and tracing, subject to instrumentation and retention design.
Delivery / IaC	GitHub-based CI/CD; Terraform	Repeatable deployments and version-controlled infrastructure; secure pipeline and state management are required.

6. Network and Security Design

- Use a dedicated VPC spanning at least two Availability Zones for the initial high-availability design.

- Place the public-facing ALB in public subnets; ECS tasks in private application subnets; and RDS in private database subnets with no direct public access.
- Restrict security-group ingress to required source security groups and ports; avoid broad inbound rules.
- Use least-privilege IAM roles for tasks and deployment identities. Avoid hard-coded credentials.
- Store secrets in a managed secret store and define rotation according to risk and service capability.
- Encrypt traffic in transit and supported data at rest; define key ownership, access, and rotation.
- Choose NAT gateways and/or VPC endpoints based on required egress, availability, traffic, and cost.

These are proposed controls; verify deployed configuration and test them before describing them as implemented.

7. Application, Database, and Cache Design

Keep the application tier stateless where practical so tasks can scale horizontally and be replaced without losing durable business state. Externalize session state only when application consistency and security requirements permit it.

RDS for PostgreSQL is the proposed authoritative relational data store. Define schema migration procedures, connection limits, backup retention, point-in-time recovery, and tested restore procedures. Use a separate read replica or reporting architecture if read scaling is required; do not assume a standard Multi-AZ standby is a read endpoint.

ElastiCache may reduce repeated reads or store suitable ephemeral data. Cache eviction or loss must not cause loss of durable business state or bypass authorization. Define TTLs and invalidation behavior for each cached object.

8. Availability, Recovery, and Observability

The proposed design uses multiple Availability Zones for the load balancer, application tasks, and database high availability. Multi-AZ improves resilience but does not, by itself, prove a particular end-to-end availability percentage.

Earlier working notes included RTO ≤1 hour and RPO ≤15 minutes as intended targets. These remain proposed objectives until the business impact is understood and backup, restore, failover, and recovery tests demonstrate that they can be met.

- Monitor request count, error rate, latency percentiles, task health, CPU/memory, ALB target health, database connections and latency, cache health, and deployment failures.
- Define actionable alarms, escalation ownership, runbooks, and an incident review process.
- Test task replacement, an Availability Zone impairment, database failover, backup restoration, and bad-deployment rollback.
- Use tracing and correlation IDs to connect requests without logging secrets or sensitive payloads.

9. CI/CD and Infrastructure as Code

Proposed delivery path: code change → automated tests and static checks → dependency/image scanning → container image build → publish to Amazon ECR → deploy to ECS → health verification → automated or operator-controlled rollback.

Terraform should manage infrastructure through reviewed, version-controlled changes. Protect state files and credentials, separate environments, and require review for high-impact changes.

The exact CI/CD service, branching model, approval gates, and deployment strategy should be selected based on team workflow and risk. They are not represented as already implemented.

10. Architecture Decision Records (ADRs)

ADR-001: Use ECS Fargate for container workloads

Status: accepted as the proposed design choice. Rationale: it reduces direct host administration and provides a managed container runtime. Trade-offs include less host-level control and the need to understand task pricing, networking, startup behavior, and platform constraints. Validate with workload tests and cost modelling.

ADR-002: Place application tasks in private subnets

Status: proposed. Application tasks should not need direct inbound access from the public internet. Design outbound access deliberately using NAT gateways, VPC endpoints, or other approved paths.

ADR-003: Use RDS PostgreSQL Multi-AZ for database high availability

Status: proposed. Managed failover can improve resilience. Trade-offs include failover interruption and cost. Define recovery objectives and test application reconnection and transaction recovery.

ADR-004: Use a cache only for cacheable data

Status: proposed. Caching can reduce repeated reads and latency, but cache loss must not compromise durable records, authorization, or correctness.

11. Failure Scenarios and Responses

Failure	Expected response / validation
ECS task crashes	Remove unhealthy targets; replace tasks; verify graceful shutdown and idempotent request handling.
One AZ impaired	Verify remaining AZ capacity can handle required load; test dependencies and egress paths.
Database failover	Reconnect through the managed endpoint with bounded retries and backoff; inspect uncertain transaction outcomes before retrying writes.
Cache unavailable	Degrade safely where possible; protect the database from a cache-miss surge; never treat cache as source of truth.
Bad deployment	Use health checks and a defined rollback process; validate database migration compatibility.
External dependency outage	Use timeouts, bounded retries, circuit breakers, and a documented recovery policy.

12. Capacity and Performance Validation

No throughput, latency, or task-count result is claimed here. Capacity depends on application behavior, task CPU/memory, database query patterns, connection pooling, cache effectiveness, and external dependencies.

- Create a representative load profile and define target throughput, p95/p99 latency, and error-rate limits.
- Load-test application and database together; observe connection saturation, slow queries, locks, CPU, memory, and scaling delay.
- Test sudden traffic increases and loss of one Availability Zone with sufficient remaining capacity.
- Test cache failure and dependency latency to understand cascading-failure risk.
- Document test environment, tool, workload, duration, results, bottlenecks, and changes.

13. Cost Model and Financial Assumptions

A reliable estimate needs a region-specific model covering task count and size, runtime hours, database instance/storage/backup configuration, load balancer usage, data transfer, NAT gateways or VPC endpoints, log ingestion and retention, cache nodes, support, and scaling.

Record assumptions, pricing date, region, utilization, and exclusions. Compare estimates with the AWS Pricing Calculator and revisit them after load tests. Do not present an estimate as actual spend.

Cost optimization should preserve availability and security requirements; the cheapest design is not necessarily the lowest-risk design.

14. Risks, Trade-offs, and Open Questions

- Workload details and business SLOs remain to be confirmed.
- The need for CloudFront in front of dynamic APIs must be justified; caching must be configured safely.
- Database sizing, connection management, backup retention, and recovery procedures require validation.
- Multi-AZ topology and egress design must be costed and tested under AZ impairment.
- Security controls, compliance requirements, and data sensitivity need a domain-specific review.
- Deployment pipeline, Terraform state protection, rollback procedures, and operational ownership require implementation.
- Capacity and cost assumptions must be supported by testing and current regional pricing.

15. Implementation Roadmap

Stage	Work	Exit evidence
1. Discovery	Confirm workload, data sensitivity, SLOs, budget, dependencies	Approved requirements and assumptions register
2. Foundation	VPC, subnetting, IAM, security groups, secrets, logging	Reviewed IaC plan and security baseline
3. Application platform	ECR, ECS/Fargate, ALB, health checks, autoscaling	Deployable service and verified health checks
4. Data layer	RDS, backups, migrations, cache if justified	Successful restore and failover tests
5. Delivery	CI/CD, tests, scanning, deployment and rollback	Repeatable pipeline run with evidence
6. Validation	Load, failure, security, recovery and cost tests	Test report, issue log, updated design
7. Operations	Dashboards, alerts, runbooks, ownership	Operational readiness review

16. Portfolio Evidence and Validation Checklist

- Publish an architecture diagram with clear trust boundaries, Availability Zones, subnets, traffic flows, and data stores.
- Include requirements, assumptions, ADRs, and trade-offs.
- Add Terraform and application code only if available and safe to publish; remove secrets, credentials, and sensitive identifiers.
- Add deployment, security, load, failover, and recovery evidence only after tests have actually been run.
- Clearly label the project as a design proposal if it has not been deployed.
- Document lessons learned, unverified items, and what would change with new requirements.

17. Professional Reflection

This case study demonstrates a structured approach to AWS architecture: separating public ingress from application and data tiers, choosing managed container and database services, considering security and observability, documenting trade-offs, and defining a validation plan.

The next maturity step is to implement a minimal version, manage it through infrastructure as code, and collect reproducible evidence. That would distinguish design competence from implementation and operational experience.

Appendix A — Suggested Repository Structure

cloudcanvas/
■■■ README.md
■■■ docs/
■ ■■■ architecture-case-study.pdf
■ ■■■ requirements-and-assumptions.md
■ ■■■ architecture-decisions/
■ ■■■ runbooks/
■■■ diagrams/
■■■ infrastructure/
■ ■■■ terraform/ (only when implementation exists)
■■■ application/ (only when implementation exists)
■■■ evidence/
■■■ validation-results/ (only for tests actually performed)

Appendix B — Publication Integrity

Before publishing, confirm that no credentials, secrets, personal data, private company material, or proprietary identifiers are present. Clearly label proposed designs and avoid claims of deployment, compliance, uptime, latency, throughput, or cost savings without evidence.